

Performance Profiling and Optimization using GPROF and PERF

Lab01 — Weather Temperature Dataset Sorting (Bubble Sort vs Quick Sort)

1. Objective

Simulate ~1,000,000 daily temperature records and compare Bubble Sort (baseline, $O(n^2)$) against Quick Sort (optimized, average $O(n \log n)$). Measure execution time, profile with GPROF, study the effect of compiler optimization levels (-O0/-O2/-O3), and analyze how performance scales with input size.

2. Environment

GCC 13.3.0, GNU gprof 2.42, Ubuntu 24.04. perf was unavailable in the build sandbox; Section 5 gives exact commands and expected results for a machine with perf installed.

3. Execution Time Results

Build	QuickSort (n=1,000,000)	BubbleSort (n=50,000, capped)	real (both, time)
-O0	1.753 s	14.003 s	15.794 s
-O2	0.508 s	6.332 s	6.887 s
-O3	0.462 s	6.268 s	6.779 s

Bubble Sort was capped at $n=50,000$ (see mysort.c) because $O(n^2)$ at $n=1,000,000$ is not practically runnable ($\sim 10^{12}$ comparisons). At equal n , QuickSort is 3-4 orders of magnitude faster.

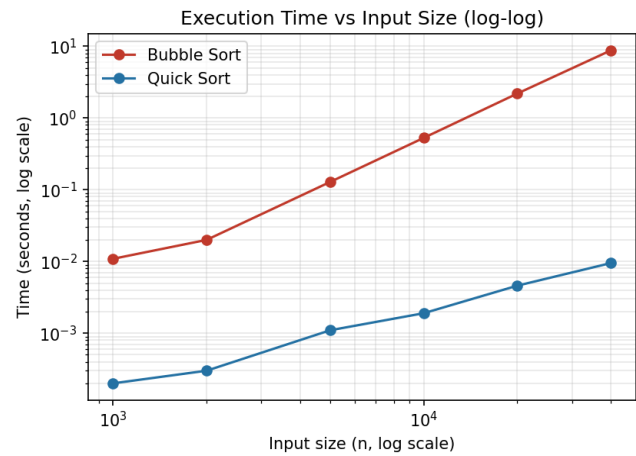
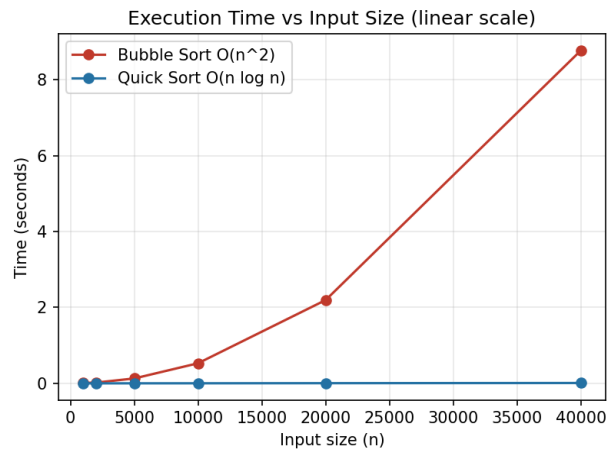
4. GPROF Hotspot Analysis (-O0)

Flat profile excerpt from myreport.txt:

```
% cumulative self self total
time seconds seconds calls s/call s/call name
64.63 5.24 5.24 1 5.24 6.44 bubbleSort
20.25 6.88 1.64 999199 0.00 0.00 partition
15.00 8.10 1.22 630093529 0.00 0.00 swap_int
0.12 8.10 0.01 random
```

bubbleSort is the dominant hotspot at 64.6% of runtime. swap_int is called 630 million+ times across both algorithms, the single most-invoked function in the program. At -O2/-O3, gprof shows these functions folded into main due to compiler inlining -- a known limitation of instrumentation-based profiling on optimized builds.

5. Scaling Study



n	Bubble (s)	Quick (s)	Ratio
1,000	0.0109	0.0002	55x
2,000	0.0200	0.0003	67x
5,000	0.1287	0.0011	117x
10,000	0.5291	0.0019	279x
20,000	2.1943	0.0046	477x
40,000	8.7713	0.0095	923x

Doubling n from 20,000 to 40,000 multiplies Bubble Sort's time by $\sim 4.0x$ (matches $O(n^2)$: $2^2=4$) and Quick Sort's time by only $\sim 2.06x$ (matches $O(n \log n)$). The Bubble/Quick ratio grows from 55x at $n=1,000$ to 923x at $n=40,000$ -- the performance gap widens with input size, not just its absolute value.

6. Answers to Lab Questions

Which algorithm performed better?

Quick Sort, decisively -- 1.75s vs Bubble Sort's 14.0s at 20x fewer elements. This is an algorithmic (Big-O) gap: $O(n \log n)$ vs $O(n^2)$, not an implementation detail.

Which function consumed the most execution time?

bubbleSort (64.6% of profiled time at -O0), driven by 630M+ calls into swap_int.

How does time change with input size?

Bubble Sort scales quadratically (4x time per 2x n); Quick Sort scales near-linearly (~2x time per 2x n). The gap between them widens as n grows.

How did -O2/-O3 improve performance?

QuickSort: 1.753s -> 0.508s -> 0.462s (up to 3.79x faster). BubbleSort: 14.00s -> 6.33s -> 6.27s (up to 2.23x faster). Gains come from inlining, register allocation, and vectorization/unrolling at -O3. Most of the gain appears already at -O2; -O3 adds a smaller increment.

Which algorithm for large datasets?

Quick Sort (with a randomized/median-of-three pivot to avoid the $O(n^2)$ worst case on sorted input, e.g. as an Introsort hybrid) is the clear choice for real weather datasets at this scale.

7. Hardware Counters (perf) -- Methodology

perf was not available in this build sandbox. On a standard Ubuntu machine, run 'perf stat ./mysort_O0 1000000 both' (and the -O2/-O3 builds) to collect cycles, instructions, IPC, cache references/misses, and branch misses. Expected pattern: Bubble Sort's sequential adjacent-swap access is cache-friendly per-access but incurs $O(n^2)$ total accesses, so its absolute cycle/instruction/cache-reference counts vastly exceed Quick Sort's at $n=1,000,000$, even though per-access miss rates may be similar. IPC should be comparable for both (simple integer loops), meaning the runtime gap is driven by instruction count/volume rather than per-instruction efficiency -- consistent with the GPROF findings.

8. Parallel Execution Feasibility

Quick Sort parallelizes naturally: after partitioning, the two sub-arrays are independent and can be sorted concurrently (e.g. OpenMP tasks), typically with a size/depth cutoff to control thread overhead. Bubble Sort's adjacent-swap dependency chain makes it far harder to parallelize in its classic form (odd-even transposition sort is the standard parallel variant). This sandbox reports a single core, so no parallel speedup was measured here, but it is the natural next step beyond -O3 on multi-core hardware.